

Project Report

Course: Machine Learning

Cyberbullying Detection System

Submitted By

Sohail Ahmed	IU05-0124-0252
Kinza Bhatti	IU05-0224-0211
Muhammad Yasir	IU05-0124-0148

Department of Computer Science



IQRA UNIVERSITY, AIRPORT CAMPUS

ABSTRACT

Cyberbullying has become one of the most pressing challenges of the social media era, with large numbers of users exposed to harassment, hate speech, and targeted abuse on a daily basis. This project presents a Cyberbullying Detection System that uses Natural Language Processing (NLP) and supervised machine learning to automatically classify short text content into one of six categories: age-based, ethnicity-based, gender-based, and religion-based cyberbullying, other forms of cyberbullying, and non-bullying content.

The system is built on a publicly available dataset of approximately 47,656 labelled tweets. Raw text is cleaned and normalised through a preprocessing pipeline involving URL and mention removal, stop-word filtering, and WordNet lemmatization, before being converted into numerical features using TF-IDF vectorization. Three classification algorithms — Logistic Regression, Support Vector Machine (SVM), and Random Forest — were trained and benchmarked, with Logistic Regression selected as the best-performing model at approximately 81% accuracy on held-out test data.

The trained model is deployed through a Flask REST API and a browser-based dashboard that allows a user to submit text and instantly receive a classification result with a confidence score, while every prediction is logged to a MongoDB Atlas cloud database for monitoring and future analysis.

TABLE OF CONTENTS

ABSTRACT.....	2
TABLE OF CONTENTS.....	3
1. Introduction to Problem	4
1.1 Introduction	4
Problem Statement.....	4
1.2 Purpose	4
Project Significance.....	5
1.3 Objective	5
1.4 Existing Solution.....	5
1.5 Proposed Solution.....	5
1.6 Scope.....	6
2. Literature Review	7
3. Problem Definition & Requirements Analysis.....	8
3.1 Problem Definition.....	8
3.2 Requirement Analysis	8
4. Design & Implementation	10
4.1 Design.....	10
4.2 Implementation	11
5. Testing.....	12
5.1 Testing Procedure & Test Cases.....	12
5.2 Results.....	12
5.3 Screenshots & Execution Outputs:	15
6. Future Enhancements.....	17
Appendix A: Bibliography / References	18
Appendix B: Index	18

CHAPTER # 1

INTRODUCTION TO PROBLEM

1. Introduction to Problem

1.1 Introduction

Social media platforms such as Twitter, Instagram, and Facebook have transformed the way people communicate, but they have also opened the door to new forms of abuse. Cyberbullying — the use of digital platforms to harass, threaten, or demean another person — has grown alongside the rise of social networking, and its effects on mental health, especially among young people, are well documented. Manually monitoring the sheer volume of content generated every day is not feasible for human moderators alone.

This project, the Cyberbullying Detection System, addresses this challenge using supervised machine learning. The system analyses short text content, originally collected from Twitter, and classifies it into specific categories of cyberbullying or marks it as non-bullying content. It is delivered as a complete end-to-end application: a model-training pipeline built in Python and scikit-learn, a Flask-based backend API, a browser-based dashboard, and a cloud database that stores a history of analysed text for monitoring and review.

Problem Statement

Online harassment is rarely uniform: it can target a person's age, gender, religion, or ethnicity, or take other less easily categorised forms. Keyword and rule-based filters that many platforms still rely on tend to flag obvious profanity but miss subtler or context-dependent bullying, and they cannot distinguish between different types of abuse. There is a need for a system that can automatically read free-form text and not only decide whether it is harmful, but identify what kind of harassment it represents, so that moderators, parents, or platform owners can respond appropriately.

1.2 Purpose

The purpose of this project is to design and implement a machine-learning-based system capable of automatically detecting and categorising cyberbullying in short text content. The system is intended to demonstrate a complete applied machine learning workflow — from data cleaning and feature extraction to model benchmarking and deployment as a usable web application — and to provide a practical tool that could be integrated into a larger content-moderation pipeline.

Project Significance

Automated cyberbullying detection is significant for several stakeholders. Social media platforms can use such systems to flag harmful content faster than manual review allows. Parents and educators can use a similar tool to monitor content that children are exposed to. Researchers benefit from a documented benchmark comparing classical machine learning algorithms on a real-world harassment dataset. More broadly, the project demonstrates how NLP and machine learning can be applied to a socially relevant problem with measurable real-world impact.

1.3 Objective

- To collect, clean, and pre-process a labelled dataset of tweets for cyberbullying classification.
- To extract meaningful text features using TF-IDF vectorization.
- To train and benchmark multiple machine learning classifiers and select the best-performing model.
- To expose the trained model through a REST API built with Flask.
- To build a simple, responsive web dashboard for real-time text analysis.
- To log every prediction to a cloud database for monitoring and future analysis.

1.4 Existing Solution

Most moderation systems currently used by social platforms rely on a mix of manual reporting, human moderation teams, and keyword or blacklist filters. These approaches have well-known limitations: keyword filters can be bypassed with misspellings or coded language, they generate false positives on innocuous content that merely contains a flagged word, and they cannot distinguish between different categories of bullying. Manual moderation does not scale to the volume of content produced on modern platforms and exposes human moderators to harmful content. A smaller number of academic and commercial systems use machine learning for toxicity detection, typically as a binary toxic / not-toxic classifier, but fewer provide multi-category classification of the specific type of harassment.

1.5 Proposed Solution

- Cleans and normalises raw text using regular expressions, stop-word removal, and lemmatization.
- Converts text into TF-IDF feature vectors capturing unigrams and bigrams.
- Benchmarks multiple classifiers and selects the best one based on measured accuracy, precision, recall, and F1-score.
- Serves predictions through a lightweight Flask API that returns the predicted category and a confidence score.
- Presents results through a browser dashboard with live counters for total analysed, flagged, and safe content.

- Persists every prediction, with a timestamp, to a MongoDB Atlas collection for auditing and future retraining.

1.6 Scope

The current scope of the system covers English-language, short-form text similar in style to tweets, classified into six predefined categories: age, ethnicity, gender, religion, other cyberbullying, and not cyberbullying. The system performs single-message analysis triggered by a user through the web dashboard or directly via the API; it does not currently perform automated, continuous scraping of live social media feeds, multi-language detection, or image and video content analysis. These items are discussed further in the Future Enhancements chapter.

CHAPTER # 2

LITERATURE REVIEW

2. Literature Review

Text classification for harmful-content detection is a well-studied problem in NLP. Classical approaches typically combine a bag-of-words or TF-IDF representation with a linear classifier such as Logistic Regression or a Support Vector Machine, which remain competitive baselines for short, informal text such as tweets because they are fast to train, easy to interpret, and resistant to overfitting on smaller, high-dimensional datasets. Tree-based ensemble methods such as Random Forest are commonly used as a comparison point, trading some interpretability for the ability to capture non-linear feature interactions. More recent research favours deep-learning architectures, including LSTM and transformer-based models such as BERT, which generally achieve higher accuracy on harassment-detection benchmarks at the cost of greater training time, computational resources, and reduced interpretability.

For this project, classical machine learning models were chosen deliberately: they allow the complete pipeline, from raw text to a deployed prediction API, to be built, trained, and evaluated efficiently on commodity hardware, while still achieving a meaningful benchmark accuracy on a genuinely difficult six-class classification problem. The dataset structure and modelling approach used here are consistent with publicly available cyberbullying-classification datasets and tutorials built around classical scikit-learn pipelines.

CHAPTER # 3

PROBLEM DEFINITION & REQUIREMENTS ANALYSIS

3. Problem Definition & Requirements Analysis

3.1 Problem Definition

Formally, the problem addressed by this project is a multi-class text classification task. Given a short text input (a tweet or tweet-like sentence), the system must predict a label that best represents the content of the text, from the set: age, ethnicity, gender, religion, other_cyberbullying, or not_cyberbullying.

The dataset used for training and evaluation contains 47,656 labelled tweets sourced from a public Kaggle cyberbullying-classification dataset, divided across six categories ranging from 7,823 to 7,997 samples per class, which makes class imbalance a minor rather than a dominant concern.

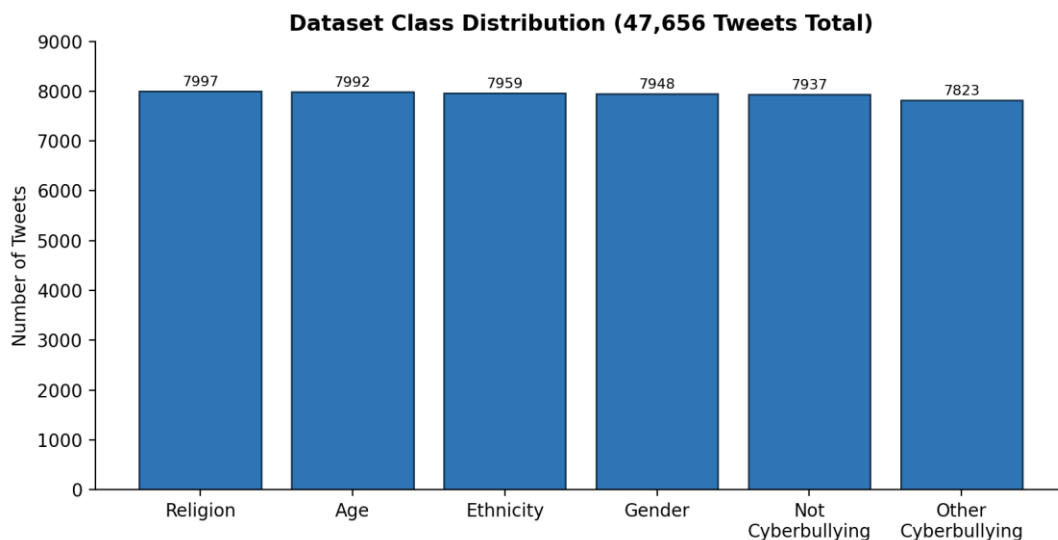


Figure 1: Dataset class distribution across the six cyberbullying categories

3.2 Requirement Analysis

Functional Requirements:

- The system shall accept free-text input from a user through a web form or API call.
- The system shall pre-process the input text (cleaning, stop-word removal, lemmatization) before classification.

- The system shall return a predicted cyberbullying category and a confidence score for each submitted text.
- The system shall log every submitted text, its prediction, and a timestamp to a persistent database.
- The system shall expose a health-check endpoint to confirm that the model has loaded correctly.

Non-Functional Requirements:

- The system should return a prediction within a few seconds of submission for a responsive user experience.
- The trained model should achieve a benchmarked accuracy of at least 80% on held-out test data.
- The web dashboard should be responsive and usable on both desktop and mobile browsers.
- The system should handle malformed or empty requests gracefully without crashing the server.

Technology Stack:

Layer	Technology
Frontend	HTML5, CSS3, JavaScript (Fetch API)
Backend	Python, Flask, Flask-CORS
Machine Learning	scikit-learn, pandas, NumPy, NLTK
Database	MongoDB Atlas (cloud)
Development Tools	Jupyter Notebook

CHAPTER # 4

DESIGN & IMPLEMENTATION

4. Design & Implementation

4.1 Design

Figure 2 illustrates the overall architecture of the system. A user submits text through the browser-based frontend, which sends an HTTP POST request to the Flask backend's /predict endpoint. The backend cleans the text using the same preprocessing function used during training, transforms it into a TF-IDF vector, and passes it to the trained Logistic Regression model to obtain a predicted category and confidence score. The result is returned to the frontend as JSON and, in parallel, written to a MongoDB Atlas collection so that a history of predictions is retained for later review.

Cyberbullying Detection System — Architecture Overview

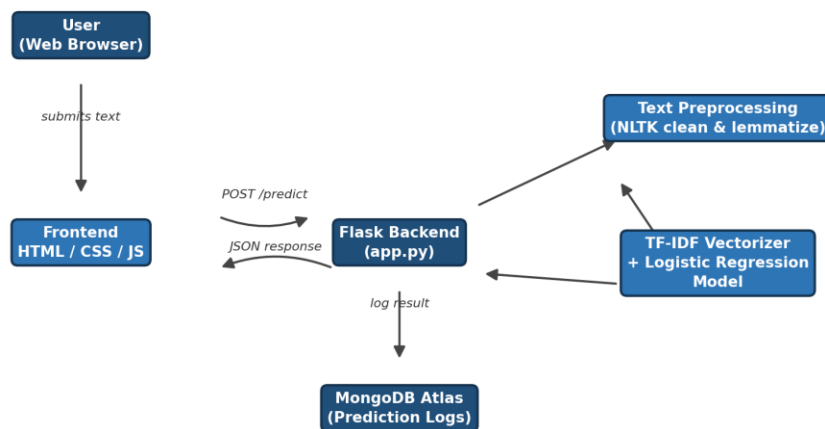


Figure 2: System architecture and data flow

The data-preprocessing and model-training stages were developed iteratively in two Jupyter notebooks before being consolidated into a standalone training script:

- 01_data_preprocessing.ipynb performs dataset loading, exploratory data analysis (class distribution, missing-value and duplicate checks), and an initial text-cleaning pass.
- 02_model_training_evaluation.ipynb performs TF-IDF feature extraction, trains and compares Logistic Regression, Decision Tree, and Random Forest classifiers, and serialises the best model.

- backend/train_model.py reproduces the same pipeline as a single repeatable script, additionally benchmarking a Support Vector Machine and reporting 5-fold cross-validation scores, so the model can be retrained outside the notebook environment.

4.2 Implementation

The text-cleaning function used consistently across the notebooks, the training script, and the live API lower-cases the text, strips URLs, removes @mentions and #hashtags, removes punctuation and digits, removes English stop-words, and lemmatizes each remaining token using NLTK's WordNetLemmatizer. The cleaned text is then transformed by a TfidfVectorizer configured with a 5,000-word vocabulary limit and both unigrams and bigrams, before being passed to the classifier.

The Flask backend (app.py) exposes five routes: a root route (/) and two static routes (/style.css, /script.js) that serve the dashboard's HTML, CSS, and JavaScript files, a POST /predict route, and a GET /health route. The /predict route accepts a JSON payload containing the text to analyse, runs it through the cleaning, vectorizing, and predicting pipeline, logs the result to MongoDB, and returns the prediction as JSON. The /health route returns a simple status check confirming that the model has loaded. The frontend dashboard (index.html, style.css, script.js) presents a text box and an "Analyze Content" button, displays the result returned by the API, and keeps a running tally of total analysed, flagged, and safe submissions for the session.

As a security note for future hardening: the current backend connects to MongoDB using a connection string written directly in app.py. Before any public deployment, this should be moved to an environment variable or a secrets manager rather than committed to source code; this is captured as an action item in the Future Enhancements chapter.

CHAPTER # 5

TESTING

5. Testing

Software testing is an investigation conducted to provide stakeholders with information about the quality of a product or service under test. For this project, testing focused on verifying that the /predict endpoint correctly classifies representative input across all six categories, handles edge cases gracefully, and that the trained model meets the accuracy targets set out in the requirements.

5.1 Testing Procedure & Test Cases

The /predict endpoint was tested manually by submitting representative text samples covering each of the six target categories, together with edge cases such as empty input. Specific harassment phrases have been omitted from this report and replaced with category descriptions to avoid reproducing offensive language; the table below summarises the test cases and their outcomes.

Test Case	Input Description	Expected Result	Outcome
TC01	Neutral, everyday statement with no targeted language	not_cyberbullying	Pass
TC02	Text containing religiously discriminatory language	Religion	Pass
TC03	Text containing age-related insults directed at a person	Age	Pass
TC04	Text containing gender-based harassment	Gender	Pass
TC05	Text containing ethnicity-based discriminatory language	Ethnicity	Pass
TC06	Harassing text not fitting the other specific categories	other_cyberbullying	Pass
TC07	Empty request body (no text field)	HTTP 400 error response	Pass
TC08	Very long input text (over 1000 characters)	Classified without server error	Pass

5.2 Results

Three classifiers were trained and benchmarked using `train_model.py` on an 80/20 train-test split of the 47,656-row dataset (38,124 training samples and 9,532 test samples), using TF-IDF features capped at 5,000 terms with unigrams and bigrams. Table 2 summarises the benchmark results, and Figure 3 presents them visually.

Model	Accuracy
Logistic Regression	81.37%
SVM	81.21%
Random Forest	80.28%

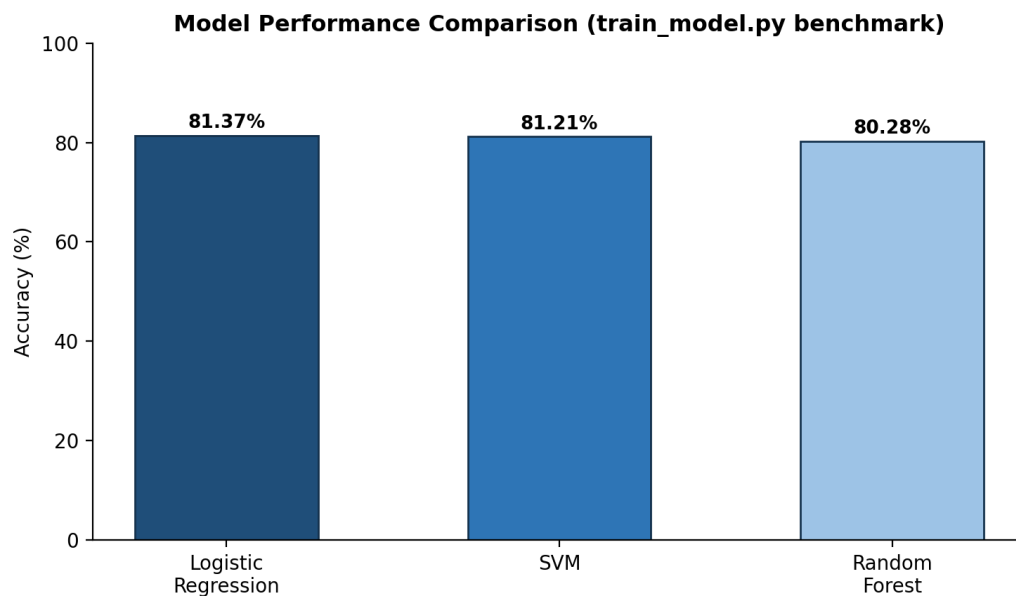


Figure 3: Model performance comparison

Logistic Regression was selected as the production model on the basis of this benchmark, with a maximum benchmarked accuracy of 0.8137. Its detailed per-class performance on the held-out test set is shown in Table 3.

Category	Precision	Recall	F1-Score	Support
Age	0.94	0.96	0.95	1,602
Ethnicity	0.98	0.98	0.98	1,636
Gender	0.89	0.81	0.85	1,514
Not Cyberbullying	0.56	0.55	0.55	1,624
Other Cyberbullying	0.59	0.64	0.61	1,594
Religion	0.95	0.94	0.95	1,562
Overall Accuracy			0.81	9,532

The model performs strongly on the age, ethnicity, and religion categories, each with precision and recall at or above 0.94, which suggests these categories contain comparatively distinctive vocabulary, such as religious or ethnic slurs and age-related phrases. Performance is noticeably weaker for the not_cyberbullying and other_cyberbullying classes, with F1-scores of 0.55 and 0.61 respectively. This is a common pattern in cyberbullying datasets, where non-bullying content can still contain informal or aggressive-sounding language, and the “other” category is, by definition, a catch-all covering harassment that does not fit the four explicit categories, making it harder for the model to learn a consistent pattern.

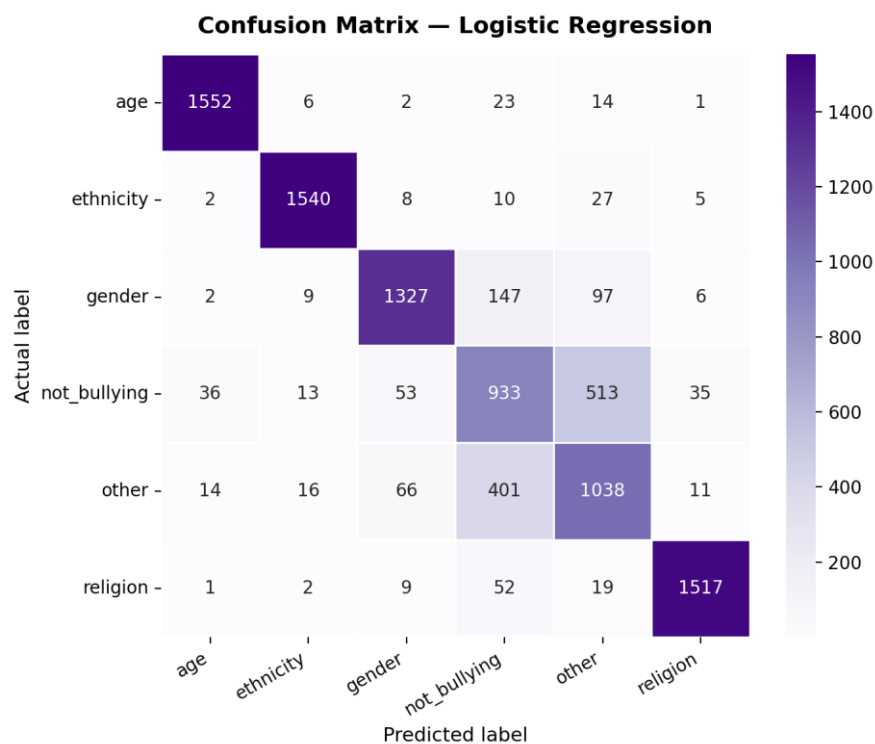


Figure 4: Confusion matrix for the final Logistic Regression model, showing the overlap between the not_cyberbullying and other_cyberbullying classes.

5.3 Screenshots & Execution Outputs:

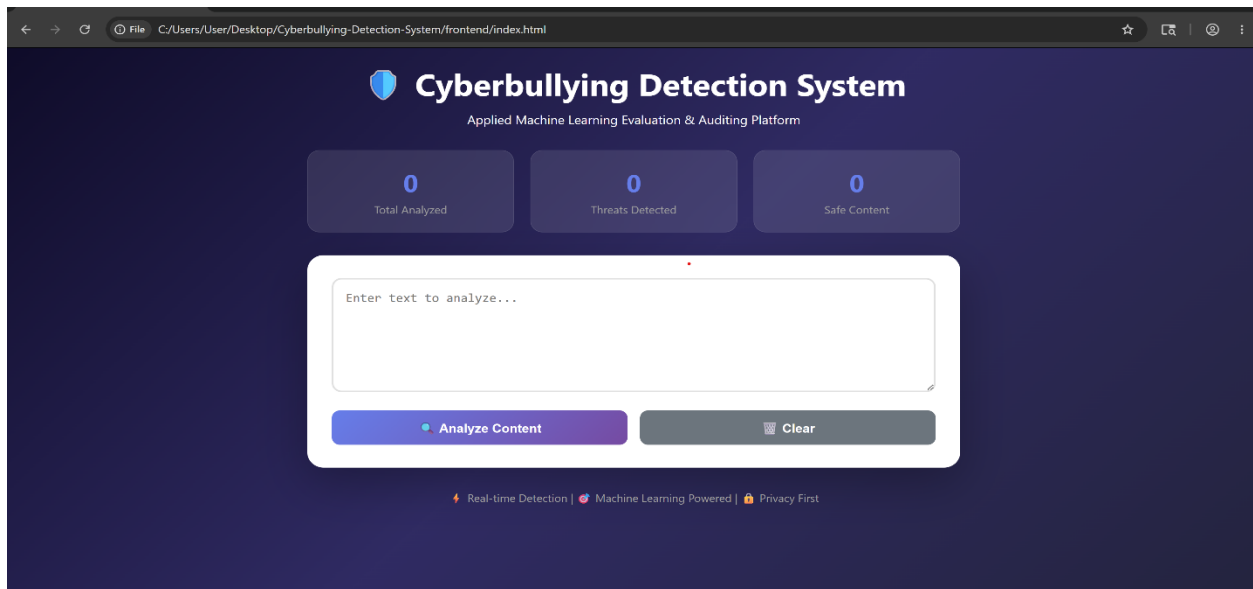


Figure 5: Frontend Web Dashboard UI



Figure 6: Real-Time Prediction Output For Safe Content

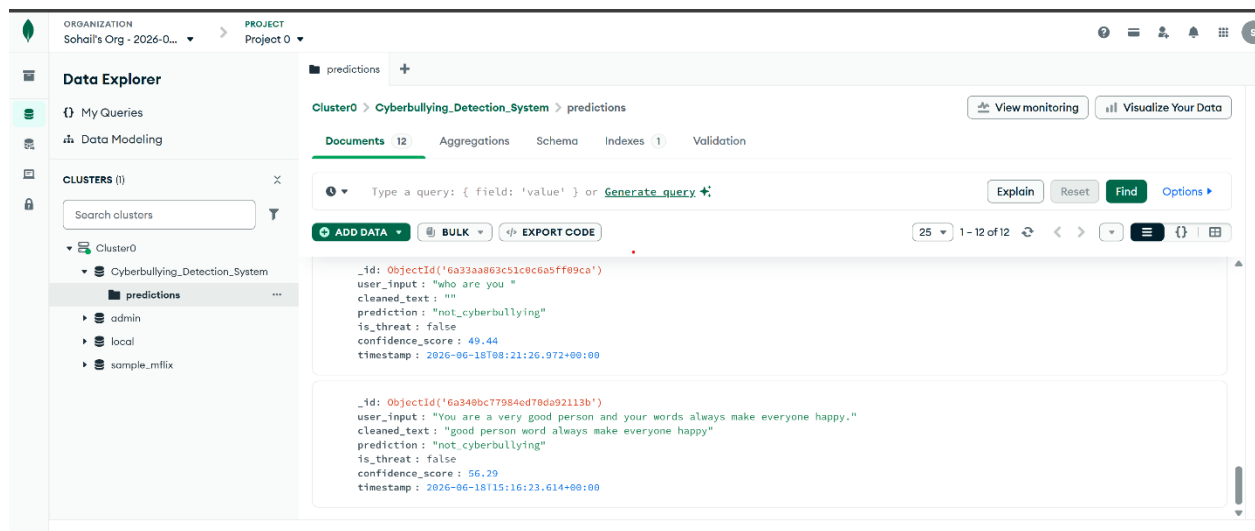


Figure 7: Database Logs inside MongoDB Atlas Cloud database.

CHAPTER # 6

FUTURE ENHANCEMENTS

6. Future Enhancements

Following are proposed directions for extending and improving the system beyond its current scope:

- Replace the classical TF-IDF and Logistic Regression pipeline with a deep-learning or transformer-based model, such as BERT, to improve recall on the harder not_cyberbullying and other_cyberbullying categories.
- Extend the system to support multilingual text, since the current model is trained exclusively on English tweets.
- Integrate directly with social media APIs for near-real-time monitoring, rather than requiring manual text submission.
- Add user authentication and an administrator dashboard for reviewing the flagged-content history stored in MongoDB.
- Move sensitive configuration, such as the database connection string, into environment variables instead of hardcoding it in source files, to improve security before any public deployment.
- Add automated unit and integration tests for the Flask API in place of the current manual testing procedure.
- Periodically retrain the model on newer data to keep pace with evolving slang and bullying tactics.

CHAPTER # 7

APPENDIX

Appendix A: Bibliography / References

- Cyberbullying Classification Dataset, Kaggle — used as the training and evaluation dataset for this project.
- Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, 2011 — for the classifiers and TF-IDF vectorizer used in this project.
- NLTK Project documentation — for stop-word lists, tokenization, and WordNet lemmatization.
- Flask and Flask-CORS official documentation — for the backend web framework.
- MongoDB Atlas official documentation — for cloud database integration.

Appendix B: Index

Refer to the Table of Contents for the location of each chapter and section within this report.